LAPORAN PRAKTIKUM PEKAN 5

PEMROGRAMAN BERORIENTASI OBJEK

“Transaksi Dinamis (Polymorphism)”

disusun Oleh:

Nabil Fikri (2511533011)

Dosen Pengampu:

Nurfiah, S.ST, M.Kom.

Asisten Praktikum:

M. Dawi Syauqi



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

A. ISI PRAKTIKUM

Praktikum Pekan 5 bertujuan untuk memahami dan menerapkan konsep **Polymorphism (Polimorfisme)** dalam Java melalui pengembangan sistem perbankan, meliputi Method Overriding, Dynamic Method Dispatch (pengikatan dinamis), penggunaan anotasi @Override, serta perancangan method yang menerima argumen polimorfik berupa objek dari berbagai subclass.

1. Refaktor Method tarikTunai pada Rekening.java

Agar fungsi penarikan dapat digunakan sebagai bagian dari fitur transfer, tipe kembalian (return type) method tarikTunai pada class Rekening diubah dari void menjadi boolean. Method mengembalikan true jika penarikan berhasil dan false jika gagal.

```
55    public boolean tarikTunai(double nominal) {
56        if (nominal <= 0) {
57            System.out.println("Gagal: Nominal tarik harus lebih dari 0!");
58            return false;
59        } else if (nominal > saldo) {
60            System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
61            return false;
62        } else {
63            saldo -= nominal;
64
65            String idTrx = "TRX-T-" + System.currentTimeMillis();
66            riwayatTransaksi.add(new Transaksi(idTrx, "Debit", nominal));
67
68            System.out.println("Tarik tunai Rp" + nominal + " berhasil. Sisa saldo: Rp" + saldo);
69            return false;
70        }
71    }
```

- Return type **boolean** memungkinkan method pemanggil (misalnya method transfer) mengetahui apakah penarikan berhasil atau tidak.
- Setiap kondisi gagal mengembalikan **false**, sedangkan penarikan yang sukses mengembalikan **true** setelah saldo dikurangi dan transaksi dicatat ke riwayatTransaksi.

2. Melakukan Overriding pada RekeningGiro.java

Pada Pekan 4, RekeningGiro masih memakai logika tarikTunai milik Rekening, yaitu saldo tidak boleh minus. Pada praktikum ini method tarikTunai ditimpa (override) di RekeningGiro agar penarikan diizinkan sampai batas overdraft.

```java
package Pekan5;

public class RekeningGiro extends Rekening {
    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal, double batasOverdraft) {
        // memanggil inisialisasi dasar dari Superclass
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }

    @Override
    public boolean tarikTunai(double nominal) {
        if (nominal <= 0) {
            System.out.println("Gagal: Nominal tarik Giro harus lebih dari 0!");
            return false;
        }

        else if (nominal > (saldo + batasOverdraft)) {
            System.out.println("Transaksi Gagal: Melebihi batas overdraft! Maksimal penarikan: Rp" + (saldo + batasOverdraft));
            return false;
        }
        else {
            saldo -= nominal;

            String idTrx = "TRX-G" + System.currentTimeMillis();
            riwayatTransaksi.add(new Transaksi(idTrx, "Debit Giro", nominal));

            System.out.println("Tarik tunai Giro Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
            return true;
        }
    }

    // getter khusus Giro
    public double getBatasOverdraft() {
        return batasOverdraft;
    }

    // catatan : penarikan hingga limit overdraft akan kita selesaikan di modul 5
}
```

- Anotasi **@Override** memberi tahu compiler bahwa method ini sengaja menimpa method milik superclass. Bila terjadi salah ketik pada nama method atau parameter, compiler langsung menampilkan error.
- Nama method, tipe parameter, dan return type **sama persis** dengan tarikTunai pada Rekening; yang berbeda hanya logika di dalamnya.
- Batas maksimal penarikan pada Giro adalah **saldo + batasOverdraft**, sehingga saldo dapat bernilai negatif.

3. Uji Coba Pengikatan Dinamis (Dynamic Method Dispatch)

Tidak ada satu baris pun kode pada Menu 3 (Tarik Tunai) di Main.java yang diubah. Pengujian dilakukan dengan membuat akun baru bertipe Giro Bisnis dengan saldo awal Rp100.000 dan batas overdraft Rp500.000, lalu menarik tunai sebesar Rp400.000.

Output :

```
Pilih Menu : 3
Masukkan PIN :
123456
Masukkan nominal tarik : 400000
Tarik tunai Giro Rp400000.0 berhasil. Saldo saat ini: Rp-300000.0
```

Jika objek tersebut adalah Rekening biasa, penarikan Rp400.000 pasti ditolak karena melebihi saldo. Namun karena objek yang tersimpan di akunAktif adalah RekeningGiro, JVM menentukan saat program berjalan (runtime) bahwa versi tarikTunai milik RekeningGiro yang harus dieksekusi. Hasilnya penarikan berhasil dan saldo menjadi **-Rp300.000**. Inilah yang disebut **Dynamic Method Dispatch** (late binding).

B. Latihan/Tugas

Tugas: Fitur Transfer Antar Rekening

1. Membuat Method transfer pada Rekening.java

Method transfer ditambahkan pada class Rekening dengan parameter polimorfik **Rekening tujuan**, sehingga dapat menerima objek RekeningTabungan maupun RekeningGiro.

```
101     public void transfer(Rekening tujuan, double nominal, String pinInput) {
102         if (!otentikasi(pinInput)) {
103             System.out.println("Transfer gagal: PIN yang Anda masukkan salah!");
104             return;
105         }
106
107         if (this.tarikTunai(nominal)) {
108             tujuan.setorTunai(nominal);
109         }
110     }
111 }
```

- Parameter **Rekening tujuan** bertipe superclass, sehingga method tidak perlu mengetahui apakah tujuan berupa Tabungan atau Giro (**argumen polimorfik**).
- **this.tarikTunai(nominal)** memanggil versi method sesuai tipe objek asal. Jika pengirim adalah RekeningGiro, aturan overdraft dipakai otomatis tanpa if-else tambahan.
- Nilai kembalian boolean dari tarikTunai diperiksa terlebih dahulu; **setorTunai** pada rekening tujuan hanya dipanggil jika penarikan sukses, sehingga tidak ada uang yang bertambah tanpa pengurangan di sisi pengirim.

2. Integrasi ke CLI Utama (Main.java): Menu 8. Transfer ke Rekening Lain

Menu baru "8. Transfer ke Rekening Lain" ditambahkan ke tampilan menu dan ke switch-case pada Main.java.

```java
            case 8:
                if (akunAktif == null) {
                    System.out.println("Anda belum memiliki rekening aktif!");
                } else {
                    System.out.print("Masukkan Nomor Rekening Tujuan: ");
                    String nomorTujuan = input.nextLine();

                    Rekening akunTujuan = null;

                    for (Rekening rekening : daftarRekening) {
                        if (rekening != null &&
                            rekening.getNomorRekening().equals(nomorTujuan)) {
                            akunTujuan = rekening;
                            break;
                        }
                    }

                    if (akunTujuan == null) {
                        System.out.println("Rekening tujuan tidak ditemukan!");
                    } else {
                        System.out.print("Masukkan Nominal Transfer: ");
                        double nominal = input.nextDouble();
                        input.nextLine();

                        System.out.print("Masukkan PIN Anda: ");
                        String pinTransfer = input.nextLine();

                        akunAktif.transfer(akunTujuan, nominal, pinTransfer);
                    }
                }
                break;
```

Output :

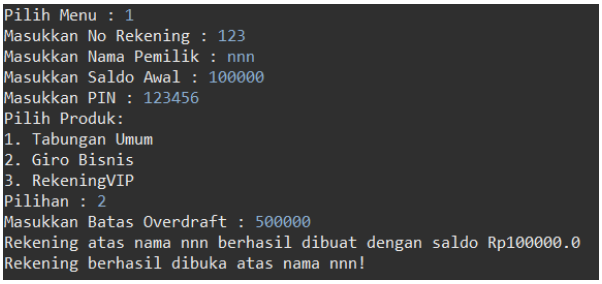
Transfer dari Tabungan ke Giro (berhasil):

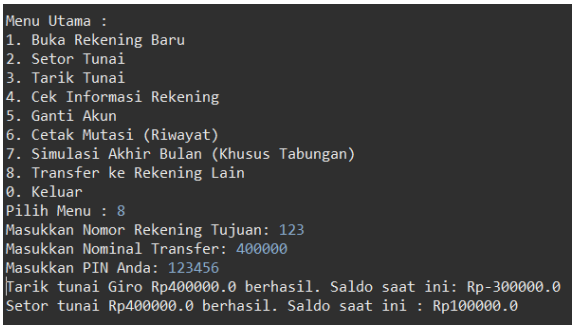
```
Pilih Menu : 1
Masukkan No Rekening : 123
Masukkan Nama Pemilik : nnn
Masukkan Saldo Awal : 500000
Masukkan PIN : 123456
Pilih Produk:
1. Tabungan Umum
2. Giro Bisnis
3. RekeningVIP
Pilihan : 1
Masukkan Suku Bunga (%) : 10
Rekening atas nama nnn berhasil dibuat dengan saldo Rp500000.0
Rekening berhasil dibuka atas nama nnn!
```

```
Pilihan : 1
Masukkan Suku Bunga (%) : 10
Rekening atas nama nnn berhasil dibuat dengan saldo Rp500000.0
Rekening berhasil dibuka atas nama nnn!

Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih Menu : 8
Masukkan Nomor Rekening Tujuan: 123
Masukkan Nominal Transfer: 300000
Masukkan PIN Anda: 123456
Tarik tunai Rp300000.0 berhasil. Sisa saldo: Rp200000.0
```

Transfer dari Giro dengan saldo minus (memakai overdraft):





- Variabel akunTujuan bertipe **Rekening** (superclass) dan dapat menampung RekeningTabungan atau RekeningGiro (relasi **Is-A**).
- Saat **akunAktif.transfer(...)** dipanggil, JVM memilih versi tarikTunai yang sesuai dengan tipe objek akunAktif sebenarnya pada saat runtime.
- Pencarian nomor rekening dilakukan dengan menelusuri **daftarRekening**; jika tidak ditemukan, program menampilkan pesan gagal dan tidak memanggil transfer.

C. Analisis dan Kesimpulan

1. Analisis

Pada praktikum Pekan 5 ini, konsep utama yang diterapkan adalah **Polymorphism (Polimorfisme)** dalam Java. Method tarikTunai pada Rekening.java direfaktor dari void menjadi boolean agar hasil penarikan dapat diperiksa oleh method lain. Kemudian RekeningGiro melakukan **Method Overriding** terhadap tarikTunai dengan anotasi @Override, sehingga batas penarikan tidak lagi sebatas saldo, melainkan saldo ditambah batasOverdraft. Dengan demikian saldo Giro dapat bernilai negatif sesuai karakteristik produknya.

Pada saat pengujian, kode Menu 3 (Tarik Tunai) tidak diubah sama sekali, tetapi perilakunya berbeda bergantung pada objek yang disimpan dalam akunAktif. Hal ini terjadi karena **Dynamic Method Dispatch**: JVM menentukan method yang dipanggil pada saat program berjalan berdasarkan tipe objek sebenarnya, bukan tipe referensinya. Pada tugas fitur transfer, method transfer menerima parameter polimorfik bertipe Rekening sehingga

dapat dipakai untuk rekening tujuan berjenis apa pun. Pemanggilan this.tarikTunai() di dalam transfer otomatis memakai aturan overdraft bila rekening pengirim adalah Giro, tanpa memerlukan struktur if-else atau switch-case untuk memeriksa jenis rekening.

2. Kesimpulan

Melalui praktikum ini dapat disimpulkan bahwa polimorfisme memungkinkan satu nama method dan satu antarmuka digunakan dengan perilaku yang berbeda sesuai tipe objek sebenarnya pada saat runtime. Method Overriding memungkinkan subclass menyesuaikan perilaku warisan dari superclass tanpa mengubah kode pemanggil, sehingga sistem lebih fleksibel, mudah diperluas, dan mengurangi percabangan kode yang berulang. Penggunaan anotasi @Override membantu mencegah kesalahan penulisan method, dan argumen polimorfik pada method transfer membuat fitur dapat bekerja untuk seluruh jenis rekening, termasuk jenis baru yang mungkin ditambahkan di masa depan.

Link Repository

https://github.com/nabilfkrr/PrakPBO_C_2511533011/tree/PrakPBO_Pekan5/Pekan5